

Natural Language Processing

Assignment-2 Report

Part 1: Working out a Toy IR System

Dataset

Consider the following three documents:

- d_1 : The star in our solar system provides heat and light.
- d_2 : That Hollywood star walked the red carpet for the movie premiere.
- d_3 : Astronomers observe distant stars and galaxies using telescopes.

Assuming the stop words are {the, in, our, and, that, for}:

- Perform tokenization.
- Remove stop words.
- Construct the inverted index for the processed documents.

After converting all terms to lowercase, tokenizing the documents and removing the given stop words:-

$$d_1 = \{\text{star, solar, system, provides, heat, light}\}$$

$$d_2 = \{\text{hollywood, star, walked, red, carpet, movie, premiere}\}$$

$$d_3 = \{\text{astronomers, observe, distant, stars, galaxies, using, telescopes}\}$$

Inverted Index

The inverted index maps each term to the list of documents in which it appears.

Term	Posting List
astronomers	$\{d_3\}$
carpet	$\{d_2\}$
distant	$\{d_3\}$
galaxies	$\{d_3\}$
heat	$\{d_1\}$
hollywood	$\{d_2\}$
light	$\{d_1\}$
movie	$\{d_2\}$
observe	$\{d_3\}$
premiere	$\{d_2\}$
provides	$\{d_1\}$
red	$\{d_2\}$
solar	$\{d_1\}$
star	$\{d_1, d_2\}$
stars	$\{d_3\}$
system	$\{d_1\}$
telescopes	$\{d_3\}$
using	$\{d_3\}$
walked	$\{d_2\}$

- Construct the TF-IDF representation for the corpus $\{d_1, d_2, d_3\}$.
- Clearly show:
 - Term Frequencies (TF)
 - Inverse Document Frequencies (IDF)
 - Final TF-IDF weights

$$TF = \frac{\text{Number of times term appears in document}}{\text{Total number of terms in document}}$$

$$IDF(t) = \ln \left(\frac{N}{df(t)} \right)$$

TF Table

Term	d_1	d_2	d_3
astronomers	0	0	1
carpet	0	1	0
distant	0	0	1
galaxies	0	0	1
heat	1	0	0
hollywood	0	1	0
light	1	0	0
movie	0	1	0
observe	0	0	1
premiere	0	1	0
provides	1	0	0
red	0	1	0
solar	1	0	0
star	1	1	0
stars	0	0	1
system	1	0	0
telescopes	0	0	1
using	0	0	1
walked	0	1	0

IDF Table

Term	Document Frequency	IDF
star	2	0.4055
All other terms	1	1.0986

TF-IDF Term-Document Matrix

Since all term frequencies are either 0 or 1, the TF-IDF value is equal to the IDF value whenever the term appears in a document.

Term	d_1	d_2	d_3
astronomers	0	0	1.0986
carpet	0	1.0986	0
distant	0	0	1.0986
galaxies	0	0	1.0986
heat	1.0986	0	0
hollywood	0	1.0986	0
light	1.0986	0	0
movie	0	1.0986	0
observe	0	0	1.0986
premiere	0	1.0986	0
provides	1.0986	0	0
red	0	1.0986	0
solar	1.0986	0	0
star	0.4055	0.4055	0
stars	0	0	1.0986
system	1.0986	0	0
telescopes	0	0	1.0986
using	0	0	1.0986
walked	0	1.0986	0

Consider the query: “star light”

- Retrieve documents using the inverted index.
- Which documents match the query terms?

The query is: $q = \text{“star light”}$
After preprocessing: $q = \{\text{star, light}\}$
From the inverted index:

$$\text{star} \rightarrow \{d_1, d_2\}$$

$$\text{light} \rightarrow \{d_1\}$$

Using OR-based Boolean retrieval:

$$\{d_1, d_2\}$$

Using AND-based Boolean retrieval:

$$\{d_1\}$$

- Represent the query using TF-IDF.
- Compute cosine similarity between the query and each document.
- Rank the documents based on similarity scores.

Answer the following:

- What is the final ranking?
- Is the ranking desirable? Justify your answer.

The query is: $q = \{\text{star, light}\}$ The non-zero query TF-IDF weights are:

$$w_{\text{star},q} = 0.4055$$

$$w_{\text{light},q} = 1.0986$$

Considering only the dimensions corresponding to *star* and *light*:

$$q = (0.4055, 1.0986)$$

Cosine similarity is:

$$\cos(q, d) = \frac{q \cdot d}{\|q\| \|d\|}$$

For d_1 :

$$\begin{aligned} q \cdot d_1 &= (0.4055)^2 + (1.0986)^2 = 1.3713 \\ \|q\| &= \sqrt{(0.4055)^2 + (1.0986)^2} = 1.1710 \\ \|d_1\| &= \sqrt{(0.4055)^2 + 5(1.0986)^2} = 2.4898 \\ \cos(q, d_1) &= \frac{1.3713}{1.1710 \times 2.4898} = 0.4704 \end{aligned}$$

For d_2 :

$$\begin{aligned} q \cdot d_2 &= (0.4055)^2 = 0.1644 \\ \|d_2\| &= \sqrt{(0.4055)^2 + 6(1.0986)^2} = 2.7214 \\ \cos(q, d_2) &= \frac{0.1644}{1.1710 \times 2.7214} = 0.0516 \end{aligned}$$

For d_3 :

$$\begin{aligned} q \cdot d_3 &= 0 \\ \cos(q, d_3) &= 0 \end{aligned}$$

Therefore, the final ranking is:

$$d_1 > d_2 > d_3$$

This ranking is desirable for the query star light because d_1 contains both query terms and uses star in the astronomical sense. Document d_2 contains star in the Hollywood sense but does not contain light. Document d_3 discusses astronomy but contains stars instead of star; without stemming, it receives zero similarity.

Consider the query: “movie star”

- Which document(s) should ideally be retrieved?
- Does the system retrieve the correct document(s)?
- Explain how word sense ambiguity (“star”) affects retrieval.

For the query: $q = \text{“movie star”}$

After preprocessing: $q = \{\text{movie, star}\}$

Ideally, the system should retrieve d_2 because it refers to a hollywood star and a movie premiere. However, star is ambiguous: it can mean a celestial body (d_1) or a celebrity (d_2). VSM matches terms lexically and does not understand word senses. While the system correctly retrieves d_2 at rank 1 due to the additional matching context word movie, document d_1 is also incorrectly matched and ranked due to the shared word star, demonstrating a limitation of lexical retrieval models.

Part 2: Building and Extending an IR System

1. Implement the retrieval component of the Information Retrieval (IR) system using the template provided. Represent documents using the TF-IDF vector space model.

Implemented the IR system using the Vector Space Model (VSM) and TF-IDF via `sklearn.feature_extraction.text.TfidfVectorizer` and `sklearn.metrics.pairwise.cosine_similarity`.

Algorithm Steps:

1. BuildIndex: Preprocessed documents (lists of sentences) are flattened into single strings. The `TfidfVectorizer` is fitted and transforms the corpus into a sparse TF-IDF matrix.
2. Rank: Queries are preprocessed, flattened, and vectorized using the fitted vectorizer.
3. Cosine similarity is computed between the query vector and all document vectors.
4. Documents are sorted in descending order of similarity.

```
def buildIndex(self, docs, docIDs):  
    """  
    builds the document index using tfidf vectors  
    """  
  
    index = None  
  
    self.docIDs = docIDs  
  
    processedDocs = []  
  
    for doc in docs:  
        flattenedDoc = []  
  
        # combining all sentences into one list  
        for sentence in doc:  
            flattenedDoc.extend(sentence)  
  
        # converting tokens back into sentence  
        processedDocs.append(" ".join(flattenedDoc))  
  
    # converts text into tfidf vectors  
    self.vectorizer = TfidfVectorizer()  
  
    index = self.vectorizer.fit_transform(processedDocs)  
  
    self.index = index
```

Figure 1: Build Index Function

```

def rank(self, queries):
    doc_IDS_ordered = []
    processedQueries = []

    for query in queries:
        flattenedQuery = []

        # flattening query sentences
        for sentence in query:
            flattenedQuery.extend(sentence)

        processedQueries.append(" ".join(flattenedQuery))

    # converting queries into tfidf vectors
    queryVectors = self.vectorizer.transform(processedQueries)

    # cosine similarity between query and docs
    similarities = cosine_similarity(queryVectors, self.index)

    for similarityScores in similarities:
        # sorting docs based on similarity
        rankedDocIndices = np.argsort(similarityScores)[::-1]

        rankedDocIDs = [self.docIDs[idx] for idx in rankedDocIndices]

        doc_IDS_ordered.append(rankedDocIDs)

    return doc_IDS_ordered

```

Figure 2: Rank Function

Part 3: Evaluating your IR System

1. Implement the following evaluation measures in the template provided:

- Precision@k
- Recall@k
- F_{0.5}-score@k
- Average Precision (AP@k)
- nDCG@k

Report the following:

- Precision@k:
- Recall@k:
- F_{0.5}-score@k:
- AP@k:
- nDCG@k:

2. Report the following global evaluation metrics:

- Mean Average Precision (MAP): Compute MAP by taking the mean of the Average Precision (AP) scores across all queries, where applicable.
- Mean Reciprocal Rank (MRR): Compute MRR by averaging the reciprocal of the rank of the first relevant document for each query, where applicable.

3. Assume that for a given query, the set of relevant documents is as listed in `cran_qrels.json`. Any document with a relevance score of 1 to 4 is considered as relevant.

- For each query in the Cranfield dataset, compute Precision, Recall, F-score, Average Precision, and nDCG for k = 1 to 10.
- Average each measure over all queries.
- Plot each evaluation metric as a function of k using the provided template code.

Report:

- Graphs of all evaluation metrics vs k
- Observations based on the plots

4. Using the `time` module in Python, report the run time of your IR system.

1. Precision@k

Precision@k measures the fraction of retrieved documents in the top k results that are relevant.

$$Precision@k = \frac{\text{Number of relevant documents retrieved in top } k}{k}$$

2. Recall@k

Recall@k measures the fraction of all relevant documents that are retrieved in the top k results.

$$Recall@k = \frac{\text{Number of relevant documents retrieved in top } k}{\text{Total number of relevant documents}}$$

3. $F_{0.5}$ -score@k

The F-score combines precision and recall into a single measure. In this assignment, $F_{0.5}$ -score is used, which gives more importance to precision than recall.

$$F_{\beta} = \frac{(1 + \beta^2) \cdot Precision \cdot Recall}{\beta^2 \cdot Precision + Recall}$$

For this assignment:

$$\beta = 0.5$$

Therefore:

$$F_{0.5} = \frac{(1 + 0.5^2) \cdot Precision \cdot Recall}{0.5^2 \cdot Precision + Recall}$$

4. Average Precision@k

Average Precision@k measures the average of precision values computed at the ranks where relevant documents are retrieved.

$$AP@k = \frac{1}{R} \sum_{i=1}^k Precision@i \cdot rel(i)$$

where:

- R is the total number of relevant documents for the query,
- $rel(i) = 1$ if the document at rank i is relevant,
- $rel(i) = 0$ otherwise.

5. Mean Average Precision

Mean Average Precision is the mean of Average Precision scores over all queries.

$$MAP@k = \frac{1}{Q} \sum_{q=1}^Q AP_q@k$$

where Q is the total number of queries.

6. nDCG@k

Normalized Discounted Cumulative Gain measures ranking quality by considering the graded relevance of documents and their positions in the ranked list.

First, Discounted Cumulative Gain is computed as:

$$DCG@k = \sum_{i=1}^k \frac{rel_i}{\log_2(i + 1)}$$

where rel_i is the relevance score of the document at rank i .

The ideal DCG is computed by sorting the relevant documents in decreasing order of their relevance scores:

$$IDCG@k = \sum_{i=1}^k \frac{rel_i^{ideal}}{\log_2(i + 1)}$$

Then nDCG is computed as:

$$nDCG@k = \frac{DCG@k}{IDCG@k}$$

If $IDCG@k = 0$, then nDCG is set to 0.

7. Mean Reciprocal Rank

Reciprocal Rank measures how early the first relevant document appears in the ranked list. For a single query:

$$RR = \frac{1}{rank}$$

where rank is the position of the first relevant document.

Mean Reciprocal Rank is computed by averaging reciprocal rank over all queries:

$$MRR@k = \frac{1}{Q} \sum_{q=1}^Q RR_q@k$$

Code Screenshot

The following screenshot shows the implementation of the evaluation metrics.

```
def queryPrecision(self, query_doc_IDs_ordered, query_id, true_doc_IDs, k):  
  
    precision = -1  
  
    #Fill in code here  
    relevantRetrieved = 0  
  
    retrievedDocs = query_doc_IDs_ordered[:k]  
  
    for docID in retrievedDocs:  
        if docID in true_doc_IDs:  
            relevantRetrieved += 1  
  
    precision = relevantRetrieved / k  
    return precision
```

Figure 3: Query Precision Function

```
def meanPrecision(self, doc_IDs_ordered, query_ids, qrels, k):  
  
    meanPrecision = -1  
  
    #Fill in code here  
    totalPrecision = 0  
  
    for i, query_id in enumerate(query_ids):  
        true_doc_IDs = []  
  
        for qrel in qrels:  
            if qrel["query_num"] == str(query_id):  
                true_doc_IDs.append(int(qrel["id"]))  
  
        totalPrecision += self.queryPrecision(  
            doc_IDs_ordered[i],  
            query_id,  
            true_doc_IDs,  
            k  
        )  
  
    meanPrecision = totalPrecision / len(query_ids)  
    return meanPrecision
```

Figure 4: Mean Precision Function

```

def queryRecall(self, query_doc_IDS_ordered, query_id, true_doc_IDS, k):

    recall = -1

    #Fill in code here
    relevantRetrieved = 0

    retrievedDocs = query_doc_IDS_ordered[:k]

    for docID in retrievedDocs:

        if docID in true_doc_IDS:
            relevantRetrieved += 1

    if len(true_doc_IDS) == 0:
        recall = 0
    else:
        recall = relevantRetrieved / len(true_doc_IDS)
    return recall

```

Figure 5: Query Recall Function

```

def meanRecall(self, doc_IDS_ordered, query_ids, qrels, k):

    meanRecall = -1

    #Fill in code here
    totalRecall = 0

    for i, query_id in enumerate(query_ids):

        true_doc_IDS = []

        for qrel in qrels:

            if qrel["query_num"] == str(query_id):
                true_doc_IDS.append(int(qrel["id"]))

        totalRecall += self.queryRecall(
            doc_IDS_ordered[i],
            query_id,
            true_doc_IDS,
            k
        )

    meanRecall = totalRecall / len(query_ids)
    return meanRecall

```

Figure 6: Mean Recall Function

```

def queryFscore(self, query_doc_IDS_ordered, query_id, true_doc_IDS, k):

    fscore = -1

    #Fill in code here
    beta = 0.5

    precision = self.queryPrecision(
        query_doc_IDS_ordered,
        query_id,
        true_doc_IDS,
        k
    )

    recall = self.queryRecall(
        query_doc_IDS_ordered,
        query_id,
        true_doc_IDS,
        k
    )

    if precision == 0 and recall == 0:
        fscore = 0
    else:
        betaSquared = beta ** 2

        fscore = ((1 + betaSquared) * precision * recall) / \
            ((betaSquared * precision) + recall)
    return fscore

```

Figure 7: Query Fscore function

```

def meanFscore(self, doc_IDS_ordered, query_ids, qrels, k):

    meanFscore = -1

    #Fill in code here
    totalFscore = 0

    for i, query_id in enumerate(query_ids):

        true_doc_IDS = []

        for qrel in qrels:

            if qrel["query_num"] == str(query_id):
                true_doc_IDS.append(int(qrel["id"]))

        totalFscore += self.queryFscore(
            doc_IDS_ordered[i],
            query_id,
            true_doc_IDS,
            k
        )

    meanFscore = totalFscore / len(query_ids)
    return meanFscore

```

Figure 8: Mean Fscore Function

```

def queryNDCG(self, query_doc_IDs_ordered, query_id, true_doc_IDs, k):

    nDCG = -1

    #Fill in code here

    retrievedDocs = query_doc_IDs_ordered[:k]

    # Build relevance dictionary
    relevanceDict = {}

    for item in true_doc_IDs:
        relevanceDict[item[0]] = item[1]

    # Compute DCG
    DCG = 0

    for i, docID in enumerate(retrievedDocs):
        relevance = relevanceDict.get(docID, 0)
        DCG += relevance / math.log2(i + 2)

    # Compute IDCG
    idealRelevances = sorted(relevanceDict.values(), reverse=True)[:k]

    IDCG = 0

    for i, relevance in enumerate(idealRelevances):
        IDCG += relevance / math.log2(i + 2)

    if IDCG == 0:
        nDCG = 0
    else:
        nDCG = DCG / IDCG

    return nDCG

```

Figure 9: Query nDCG Function

```

def meanNDCG(self, doc_IDS_ordered, query_ids, qrels, k):

    meanNDCG = -1

    #Fill in code here
    totalNDCG = 0

    for i, query_id in enumerate(query_ids):

        true_doc_IDS = []

        for qrel in qrels:

            if qrel["query_num"] == str(query_id):

                true_doc_IDS.append(
                    (int(qrel["id"]), int(qrel["position"])))

        totalNDCG += self.queryNDCG(
            doc_IDS_ordered[i],
            query_id,
            true_doc_IDS,
            k
        )

    meanNDCG = totalNDCG / len(query_ids)
    return meanNDCG

```

Figure 10: Mean nDCG Function

```

def queryAveragePrecision(self, query_doc_IDS_ordered, query_id, true_doc_IDS, k):

    avgPrecision = -1

    #Fill in code here
    relevantCount = 0
    precisionSum = 0

    retrievedDocs = query_doc_IDS_ordered[:k]

    for i, docID in enumerate(retrievedDocs):

        if docID in true_doc_IDS:

            relevantCount += 1

            precisionAtI = relevantCount / (i + 1)

            precisionSum += precisionAtI

    if len(true_doc_IDS) == 0:
        avgPrecision = 0
    else:
        avgPrecision = precisionSum / len(true_doc_IDS)
    return avgPrecision

```

Figure 11: Query Average Precision Function

```

def meanAveragePrecision(self, doc_IDS_ordered, query_ids, q_rels, k):

    meanAveragePrecision = -1

    #Fill in code here
    totalAP = 0

    for i, query_id in enumerate(query_ids):

        true_doc_IDS = []

        for qrel in q_rels:

            if qrel["query_num"] == str(query_id):
                true_doc_IDS.append(int(qrel["id"]))

        totalAP += self.queryAveragePrecision(
            doc_IDS_ordered[i],
            query_id,
            true_doc_IDS,
            k
        )

    meanAveragePrecision = totalAP / len(query_ids)
    return meanAveragePrecision

```

Figure 12: Mean Average Precision Function

```

def queryReciprocalRank(self, query_doc_IDS_ordered, query_id, true_doc_IDS, k):

    reciprocalRank = -1

    #Fill in code here
    retrievedDocs = query_doc_IDS_ordered[:k]

    reciprocalRank = 0

    for i, docID in enumerate(retrievedDocs):

        if docID in true_doc_IDS:

            reciprocalRank = 1 / (i + 1)

            break

    return reciprocalRank

```

Figure 13: Query Reciprocal Rank Function

```

def meanReciprocalRank(self, doc_IDS_ordered, query_ids, qrels, k):

    meanReciprocalRank = -1

    #Fill in code here
    totalRR = 0

    for i, query_id in enumerate(query_ids):

        true_doc_IDS = []

        for qrel in qrels:

            if qrel["query_num"] == str(query_id):
                true_doc_IDS.append(int(qrel["id"]))

        totalRR += self.queryReciprocalRank(
            doc_IDS_ordered[i],
            query_id,
            true_doc_IDS,
            k
        )

    meanReciprocalRank = totalRR / len(query_ids)
    return meanReciprocalRank

```

Figure 14: Mean Reciprocal Rank Function

Experimental Results

The evaluation was performed for $k = 1$ to $k = 10$. The following table reports the averaged evaluation results over all queries.

Table 1: Evaluation Metrics for the IR System

k	Precision@k	Recall@k	F-score@k	MAP@k	nDCG@k	MRR@k
1	0.6800	0.1170	0.3202	0.1170	0.5522	0.6800
2	0.5733	0.1919	0.3778	0.1850	0.5121	0.7244
3	0.5067	0.2425	0.3848	0.2269	0.4854	0.7437
4	0.4511	0.2823	0.3751	0.2541	0.4740	0.7515
5	0.4178	0.3157	0.3676	0.2734	0.4701	0.7586
6	0.3881	0.3478	0.3574	0.2908	0.4726	0.7608
7	0.3625	0.3735	0.3453	0.3040	0.4762	0.7621
8	0.3383	0.3943	0.3311	0.3149	0.4788	0.7626
9	0.3160	0.4083	0.3156	0.3213	0.4807	0.7631
10	0.2960	0.4206	0.3009	0.3267	0.4818	0.7631

Evaluation Plot

The following figure shows the variation of evaluation metrics with respect to k .

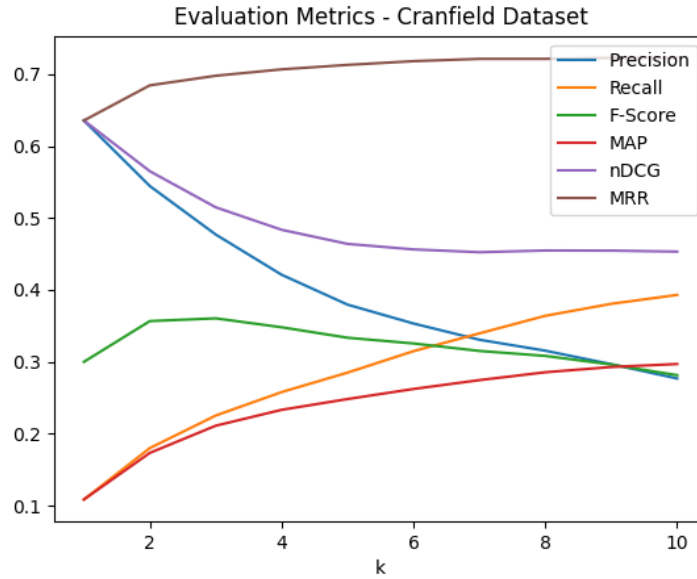


Figure 15: Evaluation Metrics

Observations

From the evaluation results, the following observations can be made:

- Precision decreases as k increases. This is expected because more documents are retrieved at larger values of k , and not all of them are relevant.
- Recall increases as k increases. This is also expected because retrieving more documents increases the chance of including relevant documents.
- $F_{0.5}$ -score balances precision and recall, but gives more importance to precision. Therefore, it is higher when precision is strong at lower ranks.
- MAP increases gradually with k , showing that relevant documents are being retrieved at different ranks in the result list.
- nDCG remains relatively stable, indicating that the ranking quality is reasonably consistent across different values of k .
- MRR is high, especially at lower values of k , which indicates that the first relevant document often appears near the top of the ranked list.

Runtime

The runtime of the IR system was measured using the `time` module in Python. The total runtime includes preprocessing, index construction, document ranking, and evaluation.

$$\text{Runtime} = \text{end_time} - \text{start_time}$$

The observed runtime of the system was:

Runtime: 13.7083 seconds

Part 4: Analysis

1. What are the limitations of the Vector Space Model (VSM)? Provide examples from the Cranfield dataset that illustrate these shortcomings in your IR system.

Your answer should include:

- Identification of key limitations of VSM
- Concrete examples from your retrieval results

The main limitations of the TF-IDF Vector Space Model (VSM) are:

- **Exact Lexical Matching:** VSM fails to recognize synonyms. E.g., a query for “aircraft” may miss documents only containing “aeroplane” or “flight”.
- **Word Sense Ambiguity (Polysemy):** VSM cannot distinguish context. A term like “flow” may refer to aerodynamics or general movement, leading to irrelevant retrievals.
- **Ignoring Word Order:** VSM uses a Bag-of-Words approach. “Pressure distribution over wing” and “Wing distribution over pressure” are treated identically.
- **Term Independence:** It ignores relationships between terms. “Shock”, “wave” and “supersonic” are strongly related in aerodynamics but modeled as entirely independent dimensions.
- **Sparse Representation:** Creates extremely high-dimensional, sparse matrices which are computationally expensive and capture little conceptual overlap.

Concrete Examples from Cranfield:

- A query about *boundary layer flow* misses relevant documents that exclusively use related terminology like *skin friction*.
- A query about *heat transfer* misses documents focusing on *thermal conduction* or *temperature distribution*.

2. The Zero-Result Problem (OOV Issue):

- What happens when a query contains a word that does not appear in the document vocabulary (Out-of-Vocabulary term)?
- How does this affect retrieval results in your system?
- Provide an example query demonstrating this issue.

The Zero-Result Problem (OOV Issue)

An Out-of-Vocabulary (OOV) term is a word in the user’s query that does not exist in the indexed document vocabulary. Because the TF-IDF vectorizer only builds dimensions for known words, OOV terms are completely ignored and do not contribute to the query vector.

Effect on Retrieval Results:

- If a query contains both known and OOV terms, the system retrieves documents based only on the known terms, often missing the core intent.
- If *all* query terms are OOV, the query becomes a zero vector. The cosine similarity with every document is exactly 0.0, resulting in zero retrieved documents.

Example Query Demonstrating the Issue:

Consider the query: $q = \text{“hypersonic aeroelastic fluttering”}$

If the specific term “aeroelastic” is absent from the dataset’s vocabulary, the system completely ignores it. If the entire phrase is unseen, the system fails entirely returning no results.